

then it must be the same as mapping the first function with the *Functor*, and then applying the second function as shown below:

```
fmap (f . g) = fmap f . fmap g
```

This can also be written for a *Functor F* as follows:

```
fmap (f . g) F = fmap f (fmap g F)
```

For example:

```
ghci> fmap (negate . abs) [1, 2, 3, 4, 5]
[-1,-2,-3,-4,-5]
```

```
ghci> fmap negate (fmap abs [1, 2, 3, 4, 5])
[-1,-2,-3,-4,-5]
```

The *Maybe* data type can also be an instance of the *Functor* class:

```
data Maybe a = Just a | Nothing
    deriving (Eq, Ord)
```

```
instance Functor Maybe where
    fmap f (Just x) = Just (f x)
    fmap f Nothing = Nothing
```

For example:

```
ghci> fmap (+2) (Nothing)
Nothing
```

```
ghci> fmap (+2) (Just 3)
Just 5
```

The two laws hold good for the *Maybe* data type:

```
ghci> id Nothing
Nothing
```

```
ghci> id Just 4
Just 4
```

```
ghci> fmap (negate . abs) (Just 4)
Just (-4)
```

```
ghci> fmap negate (fmap abs (Just 4))
Just (-4)
```

The *Applicative* type class is defined to handle cases where a function is enclosed in a *Functor*, like *Just* $(*)$:

```
class Functor f => Applicative f where
    -- | Lift a value.
```

```
pure :: a -> f a
```

```
-- | Sequential application.
(<*>) :: f (a -> b) -> f a -> f b
```

The ' $\$>$ ' is defined as a synonym for '*fmap*':

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
f <$> a = fmap f a
```

The *Applicative Functor* must also satisfy a few mathematical laws. The *Maybe* data type can be an instance of the *Applicative* class:

```
instance Applicative Maybe where
    pure = Just
    (Just f) <*> (Just x) = Just (f x)
    _ <*> _ = Nothing
```

A few examples of *Maybe* for the *Applicative* type class are shown below:

```
ghci> import Control.Applicative
```

```
ghci> Just (+2) <*> Just 7
Just 9
```

```
ghci> (*) <$> Just 3 <*> Just 4
Just 12
```

```
ghci> min <$> Just 4 <*> Just 6
Just 4
```

```
ghci> max <$> Just "Hello" <*> Nothing
Nothing
```

```
ghci> max <$> Just "Hello" <*> Just "World"
Just "World"
```

The *Applicative Functor* unwraps the values before performing an operation.

For a data type to be an instance of the *Monoid* type class, it must satisfy two properties:

1. Identity value
2. Associative binary operator

$a * (b * c) = (a * b) * c$

These are defined in the *Monoid* type class:

```
class Monoid a where
    mempty :: a -- identity
    mappend :: a -> a -> a -- associative binary operation
```

Lists can be a *Monoid*. The identity operator is `[]` and the associative binary operator is `(++)`. The instance definition of